

PROGETTO: Pipeline E2E AWS per Analizzare le Cripto Bitcoin (BTC) e Monero (XMR), tramite Confronto Price /Sentiment Google Trends (periodo 2019 – 2024).

SCOPO: Analizziamo la correlazione tra il sentiment delle ricerche su Google e la volatilità del prezzo di una criptovaluta. Valutiamo l'entità di questa correlazione e quale dei due fattori – movimenti di prezzo o interesse online – guida la relazione, utilizzando i dati di Bitcoin (BTC) e Monero (XMR) nel periodo 2019-2024.

Attraverso ambiente cloud, è possibile utilizzare grandi volumi di dati ed eseguire una automazione completa del processo, dalla raccolta di dati grezzi alla disponibilità di informazioni strategiche, tramite sistemi di orchestrazione delle funzioni. Un ulteriore aspetto è la scalabilità del piano, che consente elaborazioni in parallelo e la possibilità estendere l'analisi ad altre criptovalute e fattori di analisi macro.

METODO E SERVIZI AWS:

- **S3:** Disponiamo di quattro file csv: BTC_History_Data, BTC_Google_Trends, XMR_History_Data, XMR_Google Trends, rispettivamente i due andamenti di prezzo di Monero e Bitcoin e i due andamenti del sentiment di ricerca, caricati in un primo ambiente denominato “bucketraw”.
- **Glue ETL - Preparazione e Unificazione dei Dati:** I file di Prezzo / Trend vengono puliti ed allineati per periodo temporale e frequenza di rilevazione. Il prezzo viene calcolato come media settimanale, in modo da allinearli alla frequenza temporale utilizzata nella misurazione della ricerca online e allo stesso tempo ridurre il “rumore” dovuto a picchi positivi e negativi del prezzo registrati su base giornaliera. Il file ripulito viene salvato in formato parquet in “bucketargnet”.
- **Redshift – Analisi dei Dati:** Le tabelle associate alle due criptovalute vengono riunite in un unico file parquet contenente: “Data” – “Price” – “IndiceGoogleTrend” all’interno di “bucketgolden” e salvato in una tabella Redshift per consentire analisi anche in altri ambienti.
- **Step Functions – Orchestrazione del Processo:** Le trasformazioni Glue ETL sono inserite in una pipeline. Le due distinte pipeline corrispondenti alle criptovalute possono eseguire anche in parallelo, massimizzando l’efficienza del processo.
- **QuickSight – Visualizzazione e Insights sui dati:** Le tabelle Bitcoin e Monero in Redshift sono pronte per query e altre elaborazioni. Vengono quindi visualizzate tramite QuickSight per cogliere immediatamente la correlazione dei due trend.

I file csv Bitcoin e Monero:

- **History_Data:** colonne “Date” – Data del valore registrato, “Price” – Prezzo di chiusura giornaliero, “Open” – Prezzo di apertura giornaliero, “High” – Prezzo massimo del giorno, “Low” – Prezzo minimo del giorno, “Vol.” – Volume degli scambi giornalieri, “Change %” – Variazione percentuale giornaliera. Usiamo le prime due colonne e puliamo i dati della colonna Prezzo con valore errato -1.

- **Google Trends,** colonne “Date” – Misurazione in settimane e “interesse criptovaluta” - Misura del trend di ricerche condotte sul motore di ricerca Google con valore da 0 a 100.

```
"Date","Price","Open","High","Low","Vol. ","Change %"  
"03/12/2024","65,619.5","65,860.0","66,135.3","65,200.0","0.19K","-0.37%"  
"03/11/2024","65,859.9","63,074.2","66,600.0","61,517.2","1.88K","4.42%"  
"03/10/2024","63,074.3","62,613.1","63,930.6","62,400.0","0.72K","0.68%"  
"03/09/2024","62,649.9","62,425.2","62,781.1","62,124.5","0.37K","0.38%"  
"03/08/2024","62,413.8","61,130.1","63,999.0","60,483.8","1.61K","2.10%"  
"03/07/2024","61,130.1","60,642.5","62,364.9","60,201.1","1.13K","0.82%"  
"03/06/2024","60,633.8","58,788.1","62,227.0","57,937.5","1.76K","3.06%"  
"03/05/2024","58,830.7","62,877.2","63,584.5","54,425.1","3.94K","-6.51%"  
"03/04/2024","62,930.5","58,214.0","63,089.9","57,580.2","3.39K","8.07%"  
"03/03/2024","58,230.2","57,223.2","58,300.0","56,650.1","0.63K","1.76%"  
"03/02/2024","57,223.2","57,564.8","57,608.4","56,900.0","0.40K","-0.59%"
```

```
"Settimana","interesse bitcoin"  
2019-03-17,9  
2019-03-24,9  
2019-03-31,15  
2019-04-07,13  
2019-04-14,11
```

Preparazione del dataset

Bucket per uso generico

Bucket di director

Bucket per uso generico (3) Info Tutte le Re

I bucket sono container per i dati archiviati in S3.

	Nome
☐	bucketargent
☐	bucketgolden
☐	bucketraw

bucketraw Info

Oggetti Proprietà Autorizzazioni Parametri

Oggetti (4)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ott
necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori info](#)

☐	Nome	Tipo
☐	BTC_EUR_Historical_Data.csv	csv
☐	google_trend_bitcoin.csv	csv
☐	google_trend_monero.csv	csv
☐	XMR_EUR Kraken Historical Data.csv	csv

Glue ETL - Notebook

“BTC Files Review”

Visualizzazione e studio per la successiva elaborazione del file “BTC History Data” tramite Glue Script:

AWS Glue Studio Notebook

You are now running a AWS Glue Studio notebook; To start using your notebook

Verifica e trasformazione del File Bitcoin History Data

```
from aws glue.context import GlueContext
from pyspark.context import SparkContext
from pyspark.sql import functions as F

glueContext = GlueContext(SparkContext.getOrCreate())
spark = glueContext.spark_session

# Definisce i path S3
input_s3_path = "s3://bucketraw/BTC_EUR_Historical_Data.csv"
output_s3_path = "s3://bucketraw/Sorted_BTC_EUR_Historical_Data.csv"

# Legge il file CSV da S3 come DataFrame
df = spark.read.csv(input_s3_path, header=True, inferSchema=True)

# Converte la colonna "Date" in tipo data
df = df.withColumn("Date", F.to_date(F.col("Date"), "MM/dd/yyyy"))

# Ordina il DataFrame in base alla colonna "Date" in ordine crescente
df_sorted = df.orderBy("Date")

# Mostra un'anteprima della tabella ordinata
df_sorted.show(20, truncate=False)

# Scrive il DataFrame ordinato di nuovo come CSV su S3
df_sorted.write.csv(output_s3_path, header=True, mode="overwrite")
```

Date	Price	Open	High	Low	Vol.	Change %
2019-01-12	3,168.1	3,175.0	3,193.2	3,130.0	2.36K	-0.21%
2019-01-13	3,074.0	3,168.1	3,188.0	3,052.2	3.52K	-2.97%
2019-01-14	3,197.6	3,075.0	3,238.8	3,068.1	7.29K	4.02%
2019-01-15	3,149.2	3,197.6	3,214.3	3,119.8	5.36K	-1.51%
2019-01-16	3,164.7	3,149.2	3,228.9	3,140.2	5.52K	0.49%
2019-01-17	3,196.5	3,167.8	3,213.3	3,123.7	6.44K	1.00%
2019-01-18	3,179.9	3,196.5	3,200.0	3,156.9	4.25K	-0.52%
2019-01-19	3,245.3	3,179.9	3,311.6	3,177.1	3.59K	2.06%
2019-01-20	3,110.4	3,245.3	3,262.3	3,062.0	4.03K	-4.16%
2019-01-21	3,112.7	3,113.8	3,148.7	3,077.4	4.53K	0.07%
2019-01-22	3,149.4	3,112.7	3,175.0	3,032.2	6.45K	1.18%
2019-01-23	3,126.6	3,149.4	3,188.1	3,102.6	5.18K	-0.72%
2019-01-24	3,158.7	3,126.6	3,181.4	3,104.8	3.53K	1.03%
2019-01-25	3,126.4	3,158.7	3,161.1	3,102.3	4.29K	-1.02%
2019-01-26	3,126.0	3,126.5	3,206.9	3,112.6	2.43K	-0.01%
2019-01-27	3,096.6	3,126.0	3,130.9	3,051.0	2.22K	-0.94%
2019-01-28	3,003.3	3,096.6	3,099.7	2,957.3	6.47K	-3.01%
2019-01-29	2,973.4	3,003.3	3,011.5	2,927.2	4.61K	-1.00%
2019-01-30	2,999.9	2,973.4	3,030.3	2,957.4	4.37K	0.89%
2019-01-31	2,978.7	2,999.9	3,025.0	2,965.3	3.88K	-0.71%

only showing top 20 rows

```

: # Filtra per visualizzare solo le colonne "Date" e "Price"
print("Tipo di dato di Date e Price:")
for col_name, col_type in df.dtypes:
    if col_name in ["Date", "Price"]:
        print(f"{col_name}: {col_type}")

Tipo di dato di Date e Price:
Date: date
Price: string

: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lag, when
from pyspark.sql.window import Window

# Mantiene solo le colonne "Date" e "Price"
df_transformed = df_sorted.drop("Open", "High", "Low", "Change %", "Vol.", "Index")

# Definisce la finestra per accedere al valore precedente
window_spec = Window.orderBy("Date")

# Sostituisce i valori -1 con il prezzo del giorno precedente
df_transformed = df_transformed.withColumn(
    "Price",
    when(col("Price") == -1, lag("Price").over(window_spec)).otherwise(col("Price"))
)

# Mostra le prime righe del DataFrame trasformato
df_transformed.show(20)

```

```

+-----+-----+
|      Date|  Price|
+-----+-----+
|2019-01-12|3,168.1|
|2019-01-13|3,074.0|
|2019-01-14|3,197.6|
|2019-01-15|3,149.2|
|2019-01-16|3,164.7|
|2019-01-17|3,196.5|
|2019-01-18|3,179.9|
|2019-01-19|3,245.3|
|2019-01-20|3,110.4|
|2019-01-21|3,112.7|
|2019-01-22|3,149.4|
|2019-01-23|3,126.6|
|2019-01-24|3,158.7|
|2019-01-25|3,126.4|
|2019-01-26|3,126.0|
|2019-01-27|3,096.6|
|2019-01-28|3,003.3|
|2019-01-29|2,973.4|
|2019-01-30|2,999.9|
|2019-01-31|2,978.7|
+-----+-----+
only showing top 20 rows

```

```

: from pyspark.sql import functions as F

# Rimuove la virgola (separatore delle migliaia) e lascia il punto decimale invariato
df_transformed = df_transformed.withColumn(
    "Price",
    F.regexp_replace(F.col("Price"), ",", "")
)

# Converte la colonna "Price" in float
df_transformed = df_transformed.withColumn(
    "Price",
    F.col("Price").cast("float")
)

# Mostra i primi 5 risultati
df_transformed.show(5, truncate=False)

# Verifica il tipo di dato della colonna "Price"
print("Tipo di dato della colonna 'Price':")
for col_name, col_type in df_transformed.dtypes:
    if col_name == "Price":
        print(f"{col_name}: {col_type}")

```

```

+-----+-----+
|Date      |Price |
+-----+-----+
|2019-01-12|3168.1|
|2019-01-13|3074.0|
|2019-01-14|3197.6|
|2019-01-15|3149.2|
|2019-01-16|3164.7|
+-----+-----+
only showing top 5 rows

Tipo di dato della colonna 'Price':
Price: float

```

```

# Tronca i decimali dopo il punto
df_transformed = df_transformed.withColumn(
    "Price",
    F.floor(F.col("Price"))
)

# Reinserisce il punto al posto della virgola dove era presente
df_transformed = df_transformed.withColumn(
    "Price",
    F.format_number(F.col("Price"), 0) # Format as integer, no decimals
)

# Mostra i risultati
df_transformed.show(10)

```

```

: from pyspark.sql.functions import col, regexp_replace

# Converte la colonna Price in stringa, sostituisce la virgola con il punto
df_transformed = df_transformed.withColumn(
    "Price", regexp_replace(col("Price").cast("string"), ",", ".")
)

# Converte Price in tipo float per calcoli futuri
df_transformed = df_transformed.withColumn(
    "Price", col("Price").cast("float")
)

# Mostra il risultato
df_transformed.show(10)

```

```

+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-12|3.168|
|2019-01-13|3.074|
|2019-01-14|3.197|
|2019-01-15|3.149|
|2019-01-16|3.164|
|2019-01-17|3.196|
|2019-01-18|3.179|
|2019-01-19|3.245|
|2019-01-20| 3.11|
|2019-01-21|3.112|
+-----+-----+
only showing top 10 rows

```

```

# Mostra le prime 5 righe
print("Prime 5 righe:")
df_transformed.show(5)

# Mostra le ultime 5 righe
print("Ultime 5 righe:")
df_transformed.orderBy(col("Date").desc()).show(5)

# Conta il numero totale di righe
num_rows = df_transformed.count()
print(f"Numero totale di righe: {num_rows}")

```

```

Ultime 5 righe:
+-----+-----+
|      Date| Price|
+-----+-----+
|2024-03-12|65.619|
|2024-03-11|65.859|
|2024-03-10|63.074|
|2024-03-09|62.649|
|2024-03-08|62.413|
+-----+-----+
only showing top 5 rows

Numero totale di righe: 1887

```

```

Prime 5 righe:
+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-12|3.168|
|2019-01-13|3.074|
|2019-01-14|3.197|
|2019-01-15|3.149|
|2019-01-16|3.164|
+-----+-----+
only showing top 5 rows

```

Visualizzazione e studio per l'elaborazione del file "Google Trends":

```

# Converte la colonna "Settimana" in tipo data
df_btccgl = df_btccgl.withColumn("Settimana", F.to_date(F.col("Settimana"), "MM/dd/yyyy"))

# Ordina il DataFrame in base alla colonna "Date" in ordine crescente
df_btccgl_sorted = df_btccgl.orderBy("Settimana")

# Mostra le prime 5 righe del DataFrame ordinato
df_btccgl_sorted.show(5)

```

```

+-----+-----+
| Settimana|interesse bitcoin|
+-----+-----+
|2019-03-17|          9|
|2019-03-24|          9|
|2019-03-31|         15|
|2019-04-07|         13|
|2019-04-14|         11|
+-----+-----+
only showing top 5 rows

```

```

# Salva il DataFrame in formato CSV su S3
df_btccgl_sorted.write.csv(output_s3_path, mode="overwrite", header=True)

```

```

# Mostra le prime 5 righe
print("Prime 5 righe:")
df_btccgl_sorted.show(5)

# Mostra le ultime 5 righe
print("Ultime 5 righe:")
df_btccgl_sorted.orderBy(col("Settimana").desc()).show(5)

# Conta il numero totale di righe
num_rows = df_btccgl_sorted.count()
print(f"Numero totale di righe: {num_rows}")

```

```

Prime 5 righe:
+-----+-----+
| Settimana|interesse bitcoin|
+-----+-----+
|2019-03-17|          9|
|2019-03-24|          9|
|2019-03-31|         15|
|2019-04-07|         13|
|2019-04-14|         11|
+-----+-----+
only showing top 5 rows

```

```

Ultime 5 righe:
+-----+-----+
| Settimana|interesse bitcoin|
+-----+-----+
|2024-03-03|         52|
|2024-02-25|         43|
|2024-02-18|         22|
|2024-02-11|         29|
|2024-02-04|         20|
+-----+-----+
only showing top 5 rows

Numero totale di righe: 260

```

Allineamento di “BTC Historical Data” e “BTC Google Trends” tramite il calcolo della media di prezzo per le settimane definite in “Google Trends” da 17-03-2019 a 03-03-2024:

```

: from pyspark.sql.functions import col, min, max, date_add, date_sub, avg, lit, dayofweek, floor

# Determina il periodo di osservazione dal dataset dei trend
min_date = df_btccgl_sorted.select(min("Settimana")).first()[0] # "2019-03-17"
max_date = df_btccgl_sorted.select(max("Settimana")).first()[0] # "2024-03-03"

# Filtra il dataset Bitcoin per il periodo definito dai trend
df_btc_filtered = df_transformed.filter(
    (col("Date") >= lit(min_date)) & (col("Date") <= lit(max_date))
)

# Verifica eventuali valori nulli nella colonna "Price"
df_btc_filtered = df_btc_filtered.filter(col("Price").isNotNull())

# Calcola la data d'inizio della settimana (giorno d'apertura: domenica)
df_btc_aligned = df_btc_filtered.withColumn(
    "Week_Start",
    date_sub(col("Date"), (dayofweek(col("Date")) - 1))
)

# Raggruppa per settimane corrette e calcola la media settimanale del prezzo
df_btc_weekly = df_btc_aligned.groupBy("Week_Start").agg(
    avg(floor(col("Price").cast("double"))).alias("Weekly_Avg_Price")
)

# Ordina le settimane
df_btc_weekly = df_btc_weekly.orderBy("Week_Start", ascending=True)

```

```

+-----+-----+
|Week_Start| Weekly_Avg_Price|
+-----+-----+
|2019-03-17|                3.0|
|2019-03-24|                3.0|
|2019-03-31| 3.7142857142857144|
|2019-04-07|                4.0|
|2019-04-14|                4.0|
|2019-04-21|                4.0|
|2019-04-28| 4.285714285714286|
|2019-05-05| 5.142857142857143|
|2019-05-12| 6.428571428571429|
|2019-05-19| 6.857142857142857|
+-----+-----+
only showing top 10 rows

```

Stessa operazione sui file di Monero, con le opportune correzioni del dato presente in Price:

```

: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lag, when
from pyspark.sql.window import Window

# Mantiene solo le colonne "Date" e "Price"
df_transformed = df_sorted.drop("Open", "High", "Low", "Change %", "Vol.", "Index")

# Definisce la finestra per accedere al valore precedente
window_spec = Window.orderBy("Date")

# Sostituisce i valori -1 con il prezzo del giorno precedente
df_transformed = df_transformed.withColumn(
    "Price",
    when(col("Price") == -1, lag("Price").over(window_spec)).otherwise(col("Price"))
)

# Mostra le prime righe del DataFrame trasformato
df_transformed.show(20)

: from pyspark.sql import functions as F

# Rimuove la virgola (separatore delle migliaia) e lascia il punto decimale invariato
df_transformed = df_transformed.withColumn(
    "Price",
    F.regexp_replace(F.col("Price"), ",", "")
)

# Converte la colonna "Price" in float
df_transformed = df_transformed.withColumn(
    "Price",
    F.col("Price").cast("float")
)

# Mostra i primi 5 risultati per vedere come sono i dati
df_transformed.show(5, truncate=False)

# Verifica il tipo di dato della colonna "Price"
print("Tipo di dato della colonna 'Price':")
for col_name, col_type in df_transformed.dtypes:
    if col_name == "Price":
        print(f"{col_name}: {col_type}")

```

```

+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-01|41.92|
|2019-01-02|45.76|
|2019-01-03|43.32|
|2019-01-04|43.98|
|2019-01-05|43.43|
|2019-01-06|48.02|
|2019-01-07|45.92|
|2019-01-08|46.17|
|2019-01-09|45.22|
|2019-01-10|39.12|
|2019-01-11|39.37|
|2019-01-12| 38.9|
|2019-01-13|37.18|
|2019-01-14|39.72|
|2019-01-15|38.98|
|2019-01-16|40.26|
|2019-01-17|39.76|
|2019-01-18|39.33|
|2019-01-19|40.27|
|2019-01-20|38.27|
+-----+-----+
only showing top 20 rows

```

Prime 5 righe:

```

+-----+-----+
|      Date|Price|
+-----+-----+
|2019-01-01|41.92|
|2019-01-02|45.76|
|2019-01-03|43.32|
|2019-01-04|43.98|
|2019-01-05|43.43|
+-----+-----+

```

Ultime 5 righe:

```

+-----+-----+
|      Date| Price|
+-----+-----+
|2024-03-12|133.29|
|2024-03-11|132.69|
|2024-03-10|133.76|
|2024-03-09| 131.4|
|2024-03-08| 134.7|
+-----+-----+
only showing top 5 rows

```

Dopo aver analizzato i csv tramite Notebook, elaboriamo il file con Script allo scopo di pulire i dati e salvarli per le successive analisi.

Modify BTC Price/Google Trends Files

[Script](#)
[Job details](#)
[Runs](#)
[Data quality](#)
[Schedules](#)
[Version Control](#)

Script [Info](#)

```

1 from aws glue.context import GlueContext
2 from pyspark.context import SparkContext
3 from pyspark.sql import functions as F
4 from pyspark.sql import SparkSession
5 from pyspark.sql.functions import col, lag, when, regexp_replace, min, max, date_sub, avg, lit, dayofweek, floor
6 from pyspark.sql.window import Window
7
8 glueContext = GlueContext(SparkContext.getOrCreate())
9 spark = glueContext.spark_session
10
11 # Percorsi S3
12 input_s3_path = "s3://bucketraw/BTC_EUR_Historical_Data.csv"
13 output_s3_path = "s3://bucketargent/Sorted_BTC_EUR_Historical_Data.parquet"
14
15 input_s3_path_trends = "s3://bucketraw/google_trend_bitcoin.csv"
16 output_s3_path_trends = "s3://bucketargent/Sorted_google_trend_bitcoin.parquet"
17
18 # Leggi i file CSV da S3
19 df = spark.read.csv(input_s3_path, header=True, inferSchema=True)
20 df_btccgl = spark.read.csv(input_s3_path_trends, header=True, inferSchema=True)
21
22 # Trasformazioni sul dataset storico BTC
23 df = df.withColumn("Date", F.to_date(F.col("Date"), "MM/dd/yyyy")).orderBy("Date")
24 df_transformed = df.drop("Open", "High", "Low", "Change %", "Vol.", "Index")
25
26 # Sostituzione valori -1 con lag
27 window_spec = Window.orderBy("Date")
28 df_transformed = df_transformed.withColumn(
29     "Price", when(col("Price") == -1, lag("Price").over(window_spec)).otherwise(col("Price"))
30 )
31 df_transformed = df_transformed.withColumn("Price", regexp_replace(col("Price"), ",", "").cast("float"))
32
33 # Filtro per il periodo osservato
34 df_btccgl = df_btccgl.withColumn("Settimana", F.to_date(F.col("Settimana"), "MM/dd/yyyy")).orderBy("Settimana")
35 min_date = df_btccgl.select(min("Settimana")).first()[0]
36 max_date = df_btccgl.select(max("Settimana")).first()[0]
37
38 df_btc_filtered = df_transformed.filter((col("Date") >= lit(min_date)) & (col("Date") <= lit(max_date)))
39 df_btc_filtered = df_btc_filtered.filter(col("Price").isNotNull())
40
41 # Calcolo settimana
42 df_btc_aligned = df_btc_filtered.withColumn("Week_Start", date_sub(col("Date"), (dayofweek(col("Date")) - 1)))
43 df_btc_weekly = df_btc_aligned.groupBy("Week_Start").agg(avg(floor(col("Price")))).alias("Weekly_Avg_Price")
44 df_btc_weekly = df_btc_weekly.orderBy("Week_Start")
45
46 # Salvataggio finale in Parquet
47 df_btc_weekly.write.mode("overwrite").parquet(output_s3_path)
48 df_btccgl.write.mode("overwrite").parquet(output_s3_path_trends)

```

AWS Glue Studio [Info](#)

Create job [Info](#)

Author in a visual interface focused on data flow.

Visual ETL

Author using an interactive code notebook.

Notebook

Author code with a script editor.

Script editor

Example jobs [Info](#)

[Create example job](#)

Your jobs (2) [Info](#)

Actions

Run job

< 1 >

⚙️

☐	Job name	Type	Created by	Last modified	AWS Glue version
☐	Modify XMR Price/Google Trends Files	Glue ETL	Script	2/11/2025, 4:22:42 PM	5.0
☐	Modify BTC Price/Google Trends Files	Glue ETL	Script	2/11/2025, 2:28:42 PM	5.0

6

I file parquet risultanti dalla pulizia vengono salvati in un bucket di pre-elaborati definito “bucketargent”. I file si possono visualizzare come segue per Bitcoin e Monero con un programma di lettura di file parquet. Verifichiamo la corretta elaborazione di prezzo medio usando le stesse settimane nei due file.

bucketargentInfo

Oggetti

Proprietà

Autorizzazioni

Parametri

Oggetti (5)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti, è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Q Trova oggetti per prefisso

☐

Nome

▲

Tipo

☐

Cleaned_XMR_EUR Kraken_Historical_Data.parquet/

Cartella

☐

Sorted_BTC_EUR_Historical_Data.parquet/

Cartella

☐

Sorted_google_trend_bitcoin.csv/

Cartella

☐

Sorted_google_trend_bitcoin.parquet/

Cartella

☐

Sorted_google_trend_monero.parquet/

Cartella

Sorted_BTC_EUR_Historical_Data.parquet/

Oggetti

Proprietà

Oggetti (1)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti, è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Q Trova oggetti per prefisso

☐

Nome

▲

Tipo

▼

☐

part-00000-fe712cb3-1c0b-4972-b7ac-95dbd9040ad3-c000.snappy.parquet

parquet

Cleaned_XMR_EUR Kraken_Historical_Data.parquet/

Oggetti

Proprietà

Oggetti (1)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti, è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Q Trova oggetti per prefisso

☐

Nome

▲

Tipo

▼

☐

part-00000-fabd68e9-417a-48c7-beeb-46841503aa97-c000.snappy.parquet

parquet

Top 10 Price Trends

Top Selling Weeks

Filters

Add Time

Add Measure

Export

Change data

Week_Start	Weekly_Avg_Price
3/17/2019,...	3,516.286
3/24/2019,...	3,556.571
3/31/2019,...	4,214.143
4/7/2019, ...	4,594
4/14/2019,...	4,629.857
4/21/2019,...	4,745.857
4/28/2019,...	4,816.429
5/5/2019, ...	5,440.714
5/12/2019,...	6,822.286
5/19/2019,...	7,126.571
5/26/2019,...	7,718.714
6/2/2019, ...	7,118.286
6/9/2019, ...	7,296
6/16/2019,...	8,496.429
6/23/2019,...	10,248.571
6/30/2019,...	9,819.571
7/7/2019, ...	10,554
7/14/2019,...	9,229.857
7/21/2019,...	8,932.857
7/28/2019,...	9,051.714
8/4/2019, ...	10,378.857

columns to build your insight

Filters

Add Time

Add Measure

Add Dimension

Week_Start	Weekly_Avg_Price
3/17/2019,...	45.857
3/24/2019,...	46.143
3/31/2019,...	56.857
4/7/2019, ...	60.143
4/14/2019,...	59.714
4/21/2019,...	58
4/28/2019,...	56.429
5/5/2019, ...	60.429
5/12/2019,...	73.714
5/19/2019,...	77.143
5/26/2019,...	83.714
6/2/2019, ...	77.571
6/9/2019, ...	78.857
6/16/2019,...	91
6/23/2019,...	92.714
6/30/2019,...	79
7/7/2019, ...	86.714
7/14/2019,...	73.714
7/21/2019,...	72.286
7/28/2019,...	73.286
8/4/2019, ...	82.571
8/11/2019,...	75.857

7

Creiamo un file unico join dei due dataset costituito da: “Date” – “Price” – “IndiceGoogleTrends”.

Bitcoin (BTC):

BTC_Join_Price_Google trend

Script

Job details

Runs

Data quality

Schedules

Version Control

Script

Info

1

from awsglue.context import GlueContext

2

from pyspark.context import SparkContext

3

from pyspark.sql import functions as F

4

from pyspark.sql import SparkSession

5

from pyspark.sql.functions import col

6

7

Inizializza Glue e Spark

8

glueContext = GlueContext(SparkContext.getOrCreate())

9

spark = glueContext.spark_session

10

11

Percorsi S3

12

input_s3_path_btc_weekly = "s3://bucketargent/Sorted_BTC_EUR_Historical_Data.parquet"

13

input_s3_path_trends = "s3://bucketargent/Sorted_google_trend_bitcoin.parquet"

14

output_s3_path_joined = "s3://bucketgolden/BTC_GoogleTrend_Joined.parquet"

15

16

Leggi i file Parquet

17

df_btc_weekly = spark.read.parquet(input_s3_path_btc_weekly)

18

df_btcggl = spark.read.parquet(input_s3_path_trends)

20

Seleziona solo le colonne necessarie

21

df_btcggl_selected = df_btcggl.select(

22

col("Settimana").alias("Week_Start"),

23

col("interesse bitcoin").alias("IndiceGoogleTrend")

24

)

25

26

Effettua il join sui campi settimanali

27

df_joined = df_btc_weekly.join(df_btcggl_selected, on="Week_Start", how="inner")

28

29

Rinomina le colonne per il formato richiesto

30

df_final = df_joined.select(

31

col("Week_Start").alias("Data"),

32

col("Weekly_Avg_Price").alias("Prezzo"),

33

col("IndiceGoogleTrend")

34

)

35

36

Salva il risultato finale come Parquet

37

df_final.write.mode("overwrite").parquet(output_s3_path_joined)

38

39

print("Parquet creato con successo in bucketgolden.")

40

Run status	Retries	Start time (Local)	End time (Local)	Duration
Succeeded	0	02/11/2025 16:52:41	02/11/2025 16:54:25	1 m 36 s

Run details

Input arguments (11)

Continuous logs

Run insights

Metrics

Troubleshooting analysis

Job name	Start time (Local)	Glue version
BTC_Join_Price_Google trend	02/11/2025 16:52:41	5.0

Monero (XMR):

XMR_Join_Price_Google trend

Script | Job details | Runs | Data quality | Schedules | Version Control

Script [Info](#)

```
1 from aws glue.context import GlueContext
2 from pyspark.context import SparkContext
3 from pyspark.sql import functions as F
4 from pyspark.sql import SparkSession
5 from pyspark.sql.functions import col
6
7 # Inizializza Glue e Spark
8 glueContext = GlueContext(SparkContext.getOrCreate())
9 spark = glueContext.spark_session
10
11 # Percorsi S3
12 input_s3_path_xmr_weekly = "s3://bucketargent/Cleaned_XMR_EUR_Kraken_Historical_Data.parquet"
13 input_s3_path_trends = "s3://bucketargent/Sorted_google_trend_monero.parquet"
14 output_s3_path_joined = "s3://bucketgolden/XMR_GoogleTrend_Joined.parquet"
15
16 # Leggi i file Parquet
17 # Dataset settimanale dei prezzi di Monero
18 df_xmr_weekly = spark.read.parquet(input_s3_path_xmr_weekly)
19 # Dataset dei trend di Google per Monero
20 df_xmrggl = spark.read.parquet(input_s3_path_trends)
21
22 # Seleziona solo la colonna "Settimana" e "Monero_interesse" rinominata
23 df_xmrggl_selected = df_xmrggl.select(
24     col("Settimana").alias("Week_Start"),
25     col("Monero_interesse").alias("IndiceGoogleTrend")
26 )
27
```

```
28 # Effettua il join sui campi settimanali
29 # Week_Start è già allineata tra i due dataframe
30 df_joined = df_xmr_weekly.join(df_xmrggl_selected, on="Week_Start", how="inner")
31
32 # Rinomina le colonne per il formato richiesto
33 df_final = df_joined.select(
34     col("Week_Start").alias("Data"),
35     col("Weekly_Avg_Price").alias("Prezzo"),
36     col("IndiceGoogleTrend")
37 )
38
39 # Salva il risultato finale come Parquet
40 df_final.write.mode("overwrite").parquet(output_s3_path_joined)
41
42 print("Parquet creato con successo in bucketgolden.")
```

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPUs)	Worker typ
Succeeded	0	02/11/2025 16:57:12	02/11/2025 16:58:38	1 m 18 s	10 DPUs	G.1X

Run details

Input arguments (11)

Continuous logs

Run insights

Metrics

Troubleshooting analysis - preview

Spark UI

Job name	Start time (Local)	Glue version	Last modified on (Local)
XMR_Join_Price_Google trend	02/11/2025 16:57:12	5.0	02/11/2025 16:58:38

Per le trasformazioni, abbiamo creato due Glue jobs per criptovaluta:

AWS Glue Studio Info

Create job Info

Author in a visual interface focused on data flow. **Visual ETL**

Author using an interactive code notebook. **Notebook**

► **Example jobs** Info

Your jobs (4) Info

Filter jobs by property

☐	Job name	Type	Created by
☐	XMR_Join_Price_Google trend	Glue ETL	Script
☐	BTC_Join_Price_Google trend	Glue ETL	Script
☐	Modify XMR Price/Google Trends Files	Glue ETL	Script
☐	Modify BTC Price/Google Trends Files	Glue ETL	Script

E i file parquet risultanti dalle trasformazioni sono stati salvati nell'ultimo bucket, chiamato "bucketgolden":

bucketgolden Info

Oggetti | Proprietà | Autorizzazioni | Parametri | Gestione | P

Oggetti (2)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

☐	Nome	Tipo
☐	BTC_GoogleTrend_Joined.parquet/	Cartella
☐	XMR_GoogleTrend_Joined.parquet/	Cartella

BTC_GoogleTrend_Joined.parquet/

Oggetti | Proprietà

Oggetti (1)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

☐	Nome	Tipo
☐	part-00000-987ca16d-ddfb-4e77-a565-3462441e4ae1-c000.snappy.parquet	parquet

XMR_GoogleTrend_Joined.parquet/

Oggetti | Proprietà

Oggetti (1)

Gli oggetti sono le entità fondamentali archiviate in Amazon S3. Per ottenere un elenco di tutti gli oggetti è necessario concedere loro le autorizzazioni esplicitamente. [Ulteriori informazioni](#)

Trova oggetti per prefisso

☐	Nome	Tipo
☐	part-00000-4b60c8ae-cb63-4b0e-a752-e9108d73e5e1-c000.snappy.parquet	parquet

Visualizziamo i parquet risultanti dall'unione dei file.

Bitcoin: Data (mese/giorno/anno) – Prezzo (media per settimana) – Sentiment tramite ricerche Google (Google Trends, valori da 0 a 100).

Data	Prezzo	IndiceGoogleTrend
3/17/2019,	3,516.286	9
3/24/2019	3,556.571	9
3/31/2019	4,214.143	15
4/7/2019,	4,594	13
4/14/2019	4,629.857	11
4/21/2019	4,745.857	12
4/28/2019	4,816.429	11
5/5/2019,	5,440.714	15
5/12/2019	6,822.286	24
5/19/2019,	7,126.571	17
5/26/2019	7,718.714	19
6/2/2019,	7,118.286	16
6/9/2019,	7,296	14
6/16/2019	8,496.429	21
6/23/2019	10,248.571	34
6/30/2019	9,819.571	22
7/7/2019,	10,554	21
7/14/2019,	9,229.857	20

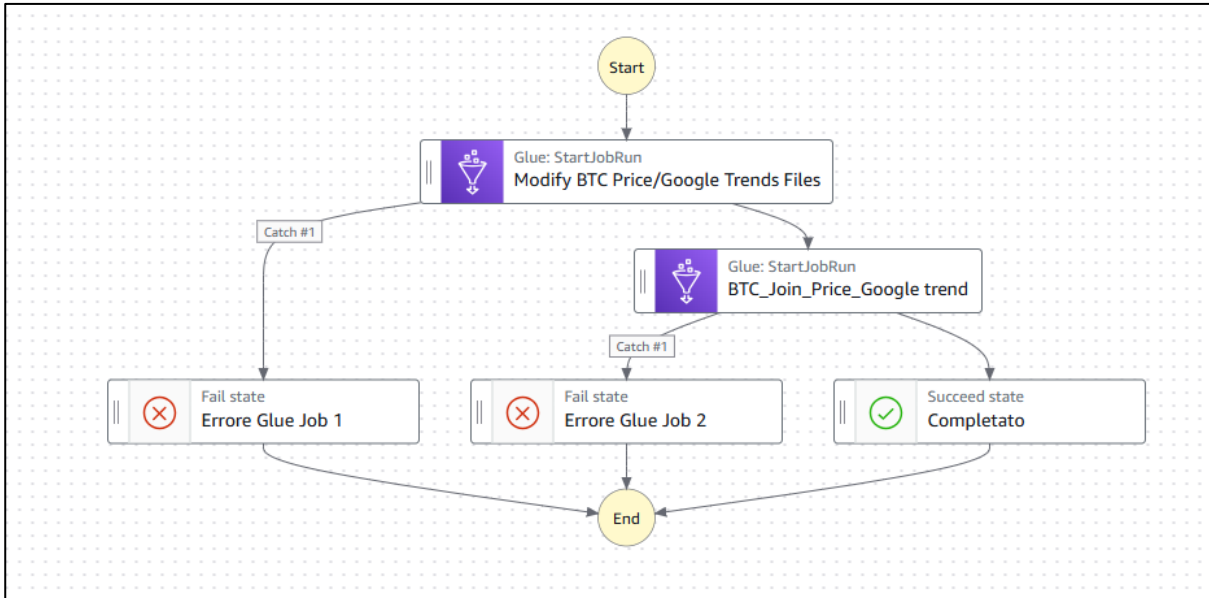
Monero: Data (mese/giorno/anno) – Prezzo (media per settimana) – Sentiment tramite ricerche Google (Google Trends, valori da 0 a 100).

Data	Prezzo	IndiceGoogleTrend
3/17/2019,	45.857	26
3/24/2019	46.143	23
3/31/2019.	56.857	28
4/7/2019,	60.143	25
4/14/2019	59.714	23
4/21/2019	58	22
4/28/2019	56.429	24
5/5/2019,	60.429	25
5/12/2019.	73.714	31
5/19/2019.	77.143	25
5/26/2019.	83.714	34
6/2/2019,	77.571	31
6/9/2019,	78.857	30
6/16/2019.	91	37
6/23/2019.	92.714	45
6/30/2019	79	35
7/7/2019, ..	86.714	32

Step Function – Pipeline per l'orchestrazione del processo.

AWS Step Functions consente di orchestrare flussi di lavoro su AWS, definendo una automazione completa della pipeline di trasformazione dei dati, dalla raccolta dei dati grezzi alla pulizia e acquisizione dei dataset. A questo fine, utilizziamo le trasformazioni Glue che abbiamo creato.

Bitcoin (BTC):



```
1 {
2   "Comment": "Step Function per eseguire due Glue ETL Jobs in s
3   "StartAt": "Modify BTC Price/Google Trends Files",
4   "States": {
5     "Modify BTC Price/Google Trends Files": {
6       "Type": "Task",
7       "Resource": "arn:aws:states:::glue:startJobRun",
8       "Parameters": {
9         "JobName": "Modify BTC Price/Google Trends Files"
10      },
11       "Next": "BTC_Join_Price_Google trend",
12       "Catch": [
13         {
14           "ErrorEquals": ["States.TaskFailed"],
15           "Next": "Errore Glue Job 1"
16         }
17       ],
18     },
19     "BTC_Join_Price_Google trend": {
20       "Type": "Task",
21       "Resource": "arn:aws:states:::glue:startJobRun",
22       "Parameters": {
23         "JobName": "BTC_Join_Price_Google trend"
24       },
25       "Next": "Completato",
26       "Catch": [
27         {
28           "ErrorEquals": ["States.TaskFailed"],
29           "Next": "Errore Glue Job 2"
30         }
31       ],
32     },
33     "Errore Glue Job 1": {
34       "Type": "Fail",
35       "Cause": "Il primo job Glue è fallito.",
36       "Error": "GlueJob1Failed"
```

```
    "ErrorEquals": ["States.TaskFailed"],
    "Next": "Errore Glue Job 1"
  }
},
"BTC_Join_Price_Google trend": {
  "Type": "Task",
  "Resource": "arn:aws:states:::glue:startJobRun",
  "Parameters": {
    "JobName": "BTC_Join_Price_Google trend"
  },
  "Next": "Completato",
  "Catch": [
    {
      "ErrorEquals": ["States.TaskFailed"],
      "Next": "Errore Glue Job 2"
    }
  ],
},
"Errore Glue Job 1": {
  "Type": "Fail",
  "Cause": "Il primo job Glue è fallito.",
  "Error": "GlueJob1Failed"
},
"Errore Glue Job 2": {
  "Type": "Fail",
  "Cause": "Il secondo job Glue è fallito.",
  "Error": "GlueJob2Failed"
},
"Completato": {
  "Type": "Succeed"
}
```

Macchine a stati (2)

Qualsiasi tipo

Nome	Tipo	Data di creazione	Stato
MyStateMachine-vd6113ex3	Standard	11 feb 2025, 21:22:26 (UTC+01:00)	Attivo

Esecuzione: 9986c25d-ca44-48b2-a5d2-b74117ff22c0

Dettagli

Input e output di esecuzione

Definizione

Stato dell'esecuzione

✓ Riuscito

▶ 1	ExecutionStarted	
▶ 2	TaskStateEntered	Modify BTC Price/Google Trends Files
▶ 3	TaskScheduled	Modify BTC Price/Google Trends Files
▶ 4	TaskStarted	Modify BTC Price/Google Trends Files
▶ 5	TaskSucceeded	Modify BTC Price/Google Trends Files
▶ 6	TaskStateExited	Modify BTC Price/Google Trends Files
▶ 7	TaskStateEntered	BTC_Join_Price_Google trend
▶ 8	TaskScheduled	BTC_Join_Price_Google trend
▶ 9	TaskStarted	BTC_Join_Price_Google trend
▶ 10	TaskSucceeded	BTC_Join_Price_Google trend
▶ 11	TaskStateExited	BTC_Join_Price_Google trend
▶ 12	SucceedStateEntered	Completato
▶ 13	SucceedStateExited	Completato
▶ 14	ExecutionSucceeded	

Componiamo un processo simile con i file Glue di Monero.

MyStateMachine-eyhwwb5z0

Progettazione
Codice

Annulla
Ripeti
Formato
Copia
Comandi
Visualizza id

```

1
2
3 "Comment": "Step Function per eseguire due Glue ETL Jobs in sequenza con gestio
4 "StartAt": "Modify XMR Price/Google Trends Files",
5 "States": {
6   "Modify XMR Price/Google Trends Files": {
7     "Type": "Task",
8     "Resource": "arn:aws:states:::glue:startJobRun",
9     "Parameters": {
10      "JobName": "Modify XMR Price/Google Trends Files"
11    },
12     "Next": "XMR_Join_Price_Google trend",
13     "Catch": [
14       {
15         "ErrorEquals": [
16           "States.TaskFailed"
17         ],
18         "Next": "Errore Glue Job 1"
19       }
20     ],
21   },
22   "XMR_Join_Price_Google trend": {
23     "Type": "Task",
24     "Resource": "arn:aws:states:::glue:startJobRun",
25     "Parameters": {
26      "JobName": "XMR_Join_Price_Google trend"
27    },
28     "Next": "Completato",
29     "Catch": [
30       {
31         "ErrorEquals": [
32           "States.TaskFailed"
33         ],
34         "Next": "Errore Glue Job 2"
35       }
36     ]
37   }
38 }

```

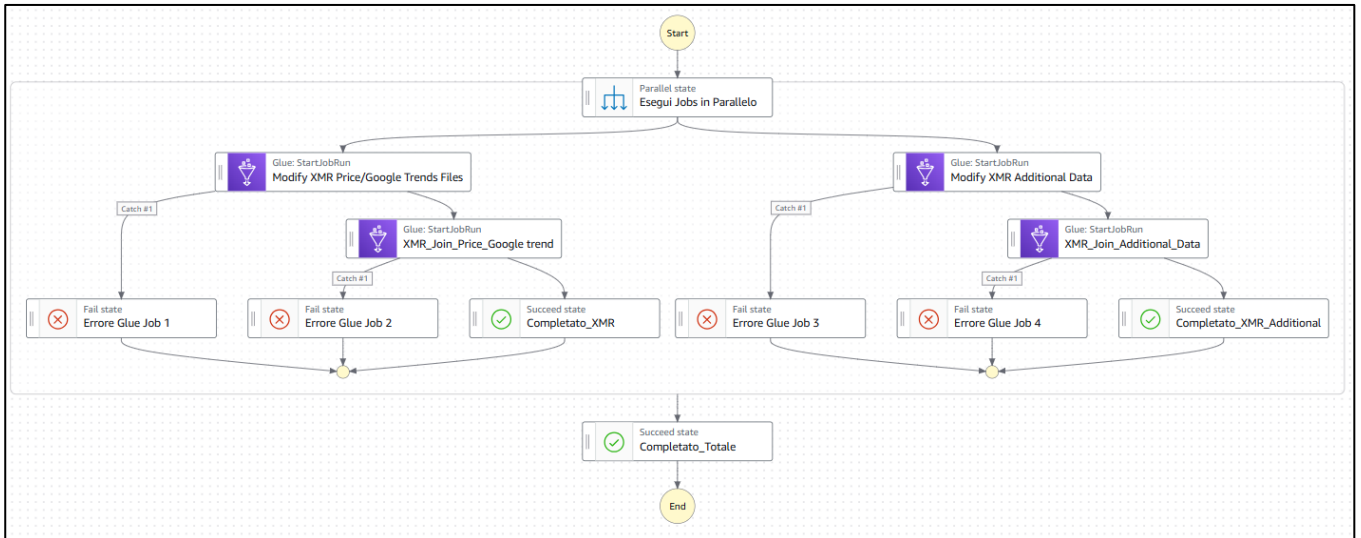
```

},
"Next": "Completato",
"Catch": [
  {
    "ErrorEquals": [
      "States.TaskFailed"
    ],
    "Next": "Errore Glue Job 2"
  }
],
"Errore Glue Job 1": {
  "Type": "Fail",
  "Cause": "Il primo job Glue è fallito.",
  "Error": "GlueJob1Failed"
},
"Errore Glue Job 2": {
  "Type": "Fail",
  "Cause": "Il secondo job Glue è fallito.",
  "Error": "GlueJob2Failed"
},
"Completato": {
  "Type": "Succeed"
}
}

```

Step Function per l'esecuzione di job in parallelo

E' possibile fare eseguire due macchine a stati in parallelo, utilizzando il tipo di stato **Parallel**. Questa funzionalità migliora l'efficienza e riduce i tempi di esecuzione.



```
"Comment": "Step Function per eseguire due Glue ETL Jobs in parallelo",
"StartAt": "Esegui Jobs in Parallelo",
"States": {
  "Esegui Jobs in Parallelo": {
    "Type": "Parallel",
    "Branches": [
      {
        "StartAt": "Modify XMR Price/Google Trends Files",
        "States": {
          "Modify XMR Price/Google Trends Files": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun",
            "Parameters": {
              "JobName": "Modify XMR Price/Google Trends Files"
            },
            "Next": "XMR_Join_Price_Google trend",
            "Catch": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "Next": "Errore Glue Job 1"
              }
            ]
          },
          "XMR_Join_Price_Google trend": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun",
            "Parameters": {
              "JobName": "XMR_Join_Price_Google trend"
            },
            "Next": "Completato_XMR",
            "Catch": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "Next": "Errore Glue Job 2"
              }
            ]
          },
          "Errore Glue Job 1": {
            "Type": "Fail",
            "Cause": "Il primo job Glue è fallito.",
            "Error": "GlueJob1Failed"
          },
          "Errore Glue Job 2": {
            "Type": "Fail",
            "Cause": "Il secondo job Glue è fallito.",
            "Error": "GlueJob2Failed"
          },
          "Completato_XMR": {
            "Type": "Succeed"
          }
        }
      },
      {
        "StartAt": "Modify XMR Additional Data",
        "States": {
          "Modify XMR Additional Data": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun",
            "Parameters": {
              "JobName": "Modify XMR Additional Data"
            },
            "Next": "XMR_Join_Additional_Data",
            "Catch": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "Next": "Errore Glue Job 3"
              }
            ]
          },
          "XMR_Join_Additional_Data": {
            "Type": "Task",
            "Resource": "arn:aws:states:::glue:startJobRun",
            "Parameters": {
              "JobName": "XMR_Join_Additional_Data"
            },
            "Next": "Completato_XMR_Additional",
            "Catch": [
              {
                "ErrorEquals": [
                  "States.TaskFailed"
                ],
                "Next": "Errore Glue Job 4"
              }
            ]
          },
          "Errore Glue Job 3": {
            "Type": "Fail",
            "Cause": "Il terzo job Glue è fallito.",
            "Error": "GlueJob3Failed"
          },
          "Errore Glue Job 4": {
            "Type": "Fail",
            "Cause": "Il quarto job Glue è fallito.",
            "Error": "GlueJob4Failed"
          },
          "Completato_XMR_Additional": {
            "Type": "Succeed"
          }
        }
      }
    ]
  },
  "Completato_Totale": {
    "Type": "Succeed"
  }
}
```

```
},
"Next": "Completato_XMR_Additional",
"Catch": [
  {
    "ErrorEquals": [
      "States.TaskFailed"
    ],
    "Next": "Errore Glue Job 4"
  }
],
"Errore Glue Job 3": {
  "Type": "Fail",
  "Cause": "Il terzo job Glue è fallito.",
  "Error": "GlueJob3Failed"
},
"Errore Glue Job 4": {
  "Type": "Fail",
  "Cause": "Il quarto job Glue è fallito.",
  "Error": "GlueJob4Failed"
},
"Completato_XMR_Additional": {
  "Type": "Succeed"
},
"Next": "Completato_Totale"
},
"Completato_Totale": {
  "Type": "Succeed"
}
```

```
},
"Next": "Completato_XMR_Additional",
"Catch": [
  {
    "ErrorEquals": [
      "States.TaskFailed"
    ],
    "Next": "Errore Glue Job 4"
  }
],
"Errore Glue Job 3": {
  "Type": "Fail",
  "Cause": "Il terzo job Glue è fallito.",
  "Error": "GlueJob3Failed"
},
"Errore Glue Job 4": {
  "Type": "Fail",
  "Cause": "Il quarto job Glue è fallito.",
  "Error": "GlueJob4Failed"
},
"Completato_XMR_Additional": {
  "Type": "Succeed"
},
"Next": "Completato_Totale"
},
"Completato_Totale": {
  "Type": "Succeed"
}
```


Redshift

Completiamo il salvataggio delle tabelle su Redshift, al fine di renderle disponibili per analisi, chiamate query e visualizzazioni in ambienti come AWS QuickSight.

Bitcoin (BTC):

The first screenshot shows a SQL query in the AWS Redshift console to load data from a Parquet file into a table named 'btc_googletrend_joined'. The query is as follows:

```
1 COPY dev.public.BTC_GoogleTrend_Joined
2 FROM 's3://bucketgolden/BTC_GoogleTrend_Joined.parquet/part-00000-987ca16d-ddfb-4e77-a565-3462441e4ae1-c000.snappy.parquet'
3 IAM_ROLE 'arn:aws:iam::423623841156:role/service-role/AmazonRedshift-CommandsAccessRole-'
4 FORMAT AS PARQUET
```

The second screenshot shows the 'Summary' section of the console, indicating that the load operation was successful. It states: 'Load into table 'btc_googletrend_joined' completed, 260 record(s) loaded successfully.' Below this, it shows 'Returned rows: 0', 'Query ID: 6082', 'Elapsed time: 12.1s', and 'Result set query:'.

The third screenshot shows a SQL query to create a table named 'bitcoin' with the following schema:

```
1 CREATE TABLE bitcoin (
2   "Date" DATE PRIMARY KEY,      -- Colonna di tipo DATE, chiave primaria
3   Price FLOAT,                  -- Colonna per il prezzo, tipo FLOAT
4   IndiceGoogleTrend DECIMAL(10, 2) -- Colonna per l'indice di Google Trend, tipo DECIMAL
5 );
```

The fourth screenshot shows the 'Summary' section for the 'CREATE TABLE' query, indicating it was successful. It shows 'Returned rows: 0', 'Elapsed time: 392ms', and 'Result set query:'.

The fifth screenshot shows a SQL query to select column names and data types from the 'information_schema.columns' table, filtered by the table name 'btc_googletrend_joined'.

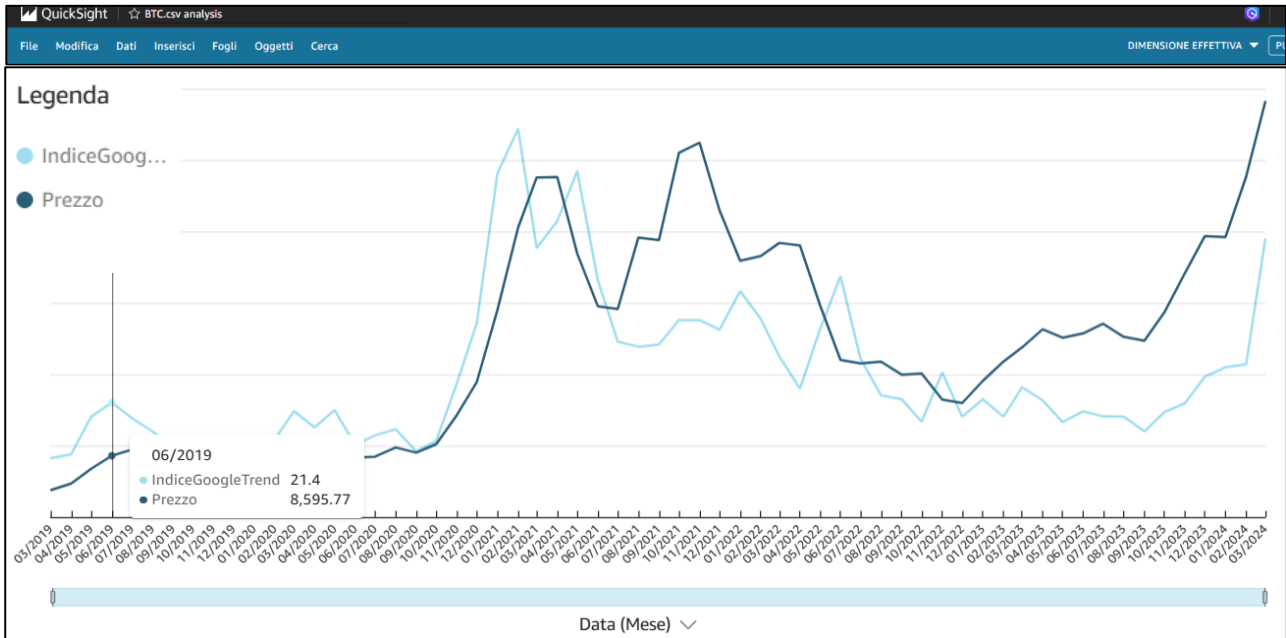
```
1 SELECT column_name, data_type
2 FROM information_schema.columns
3 WHERE table_name = 'btc_googletrend_joined';
4
```

Esegui le stesse operazioni per il file di Monero.

QuickSight

Una volta elaborato il dataset finale, è possibile utilizzarlo per creare visualizzazioni che evidenzino le tendenze nel tempo, permettendo di ottenere insight significativi.

Grafico Bitcoin (BTC):



Insights: I due trend mostrano una forte correlazione lungo tutto l'andamento delle performance. In particolare, si registra un picco storico nel periodo che inizia da settembre 2020. Successivamente, si osserva una netta discontinuità nei valori medi dei trend rispetto al periodo precedente. Il trend delle ricerche sembra anticipare con precisione l'andamento immediato del prezzo del Bitcoin e avere effetti durevoli nel tempo sul prezzo, mantenendolo su una media significativamente più elevata rispetto al periodo precedente a settembre 2020.

Partendo da metà 2022, il prezzo del Bitcoin inizia a rendersi indipendente e anzi ad assumere un ruolo guida nella relazione, trainato forse da altri fattori di mercato e di contesto, con il trend delle ricerche online che lo segue a sua volta. In un quadro più ampio, la crescita di interesse online sostenuta fino a quel momento appare ancora supportare il prezzo, mostrando come la variabile di ricerca continui a essere un fattore cruciale tra quelli utili a predire l'andamento di prezzo del Bitcoin.

Grafico Monero (XMR): La correlazione tra il trend delle ricerche (azzurro) e il trend di prezzo (blu) si mostra in modo abbastanza chiaro e lineare, per l'intero periodo osservato.

